

Integração Contínua

Aula 10 · Bloco 3 — Do Código à Produção

Integrar todo dia — e deixar uma máquina conferir cada integração

🎯 Nesta aula

1. **Integrar todo dia**, verificar toda vez — e o vermelho como prioridade
2. O **build** que dá o mesmo resultado em qualquer máquina
3. O **pipeline**: os estágios, a ordem e os dez minutos
4. A **pirâmide de testes** — e o teste instável
5. A **verificação que barra** a mudança, e a métrica que vira meta

O mesmo sábado, duas vezes

A correção do frete, da Aula 09, passou a calcular com **três casas decimais** — e o serviço de pagamento só aceita duas.



Ana não teve mais cuidado no segundo sábado. Mudou **quando o defeito foi descoberto**.

O que é integração contínua

Integrar no principal, todo dia

ramos curtos, como no GitHub Flow e no trunk-based development da Aula 09

sem isso: a máquina confere trabalho isolado

Verificar cada integração

a cada envio, uma máquina constrói o sistema e roda os testes

sem isso: depende de alguém lembrar

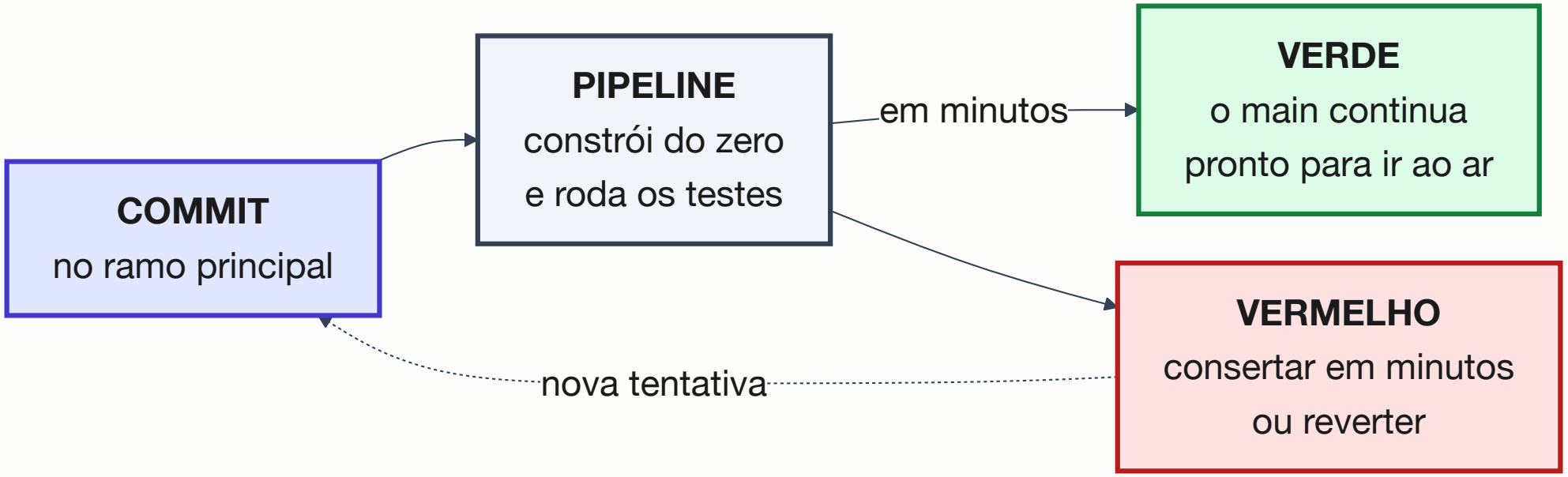
Consertar o vermelho primeiro

integração quebrada é a prioridade do time até voltar a passar

sem isso: o vermelho vira paisagem

A prática nasceu na **XP**, de Kent Beck, nos anos 1990. O texto de referência é de Martin Fowler — 2000, revisto em janeiro de 2024.

O ciclo curto de retorno



O ganho não é impedir o erro: é **descobri-lo em minutos**, quando ainda é uma linha, e não em semanas, quando são quatrocentas.

💡 A prática, não a ferramenta

Um pipeline configurado
com ramos que vivem três semanas
não é integração contínua.

É uma máquina conferindo, com pontualidade,
um trabalho que continua integrado tarde —
a **"meia integração"**, nas palavras de Fowler.

O vermelho é a prioridade

> *"Ninguém tem tarefa mais prioritária do que consertar o build."* — Kent Beck

falhou
o pipeline do `main` ficou vermelho

→

minutos
quem quebrou tenta consertar

→

não é óbvio?
reverte — o `git revert` da Aula 09

→

verde
conserta com calma no próprio ramo

> ⚠️ **Um alarme que toca o tempo todo é um alarme desligado.** Vermelho por três dias, e o time aprende a integrar sem olhar o resultado.

Por que isso importa mais agora

+25%

de adoção de IA no desenvolvimento — a referência da comparação

−1,5%

na **vazão** da entrega, estimada

−7,2%

na **estabilidade** da entrega, estimada

DORA 2024. A explicação dos autores: escrever melhor não vira entregar melhor sem o básico — **mudanças pequenas e testes robustos**.

> 💡 **DORA 2025, com quase 5 mil profissionais:** a IA é um **amplificador**. Reforça o time que já entrega bem e expõe o que não entrega. Quanto mais rápido se escreve, mais a verificação automática importa.

"Funciona na minha máquina"

Automático

um comando só, sem passo manual

`npm run build`

A partir do repositório

tudo versionado, ou baixado numa versão fixada

o lockfile da Aula 09

Em ambiente limpo

cada execução começa numa máquina nova

criada para o build, apagada no fim

Reproduzível

a mesma entrada dá o mesmo resultado

o build do `a3f9c21` sai hoje igual a ontem

O **build** é a construção do sistema — a 2ª atividade da gerência de configuração da Aula 09.

22/03/2016: onze linhas

a disputa
um autor perde para uma empresa o nome de um pacote

→

a retirada
ele tira do npm todos os seus pacotes

→

left-pad
11 linhas, dependência de dependência de Babel e React

→

horas
builds do mundo todo falham — sem ninguém mudar código

→

a volta
o npm republica o pacote e muda a regra

As dependências são itens de configuração — inclusive as que você não escolheu diretamente.

O que o build não pode deixar ao acaso

Lockfile

a versão exata de cada biblioteca, inclusive das indiretas

Cópia interna

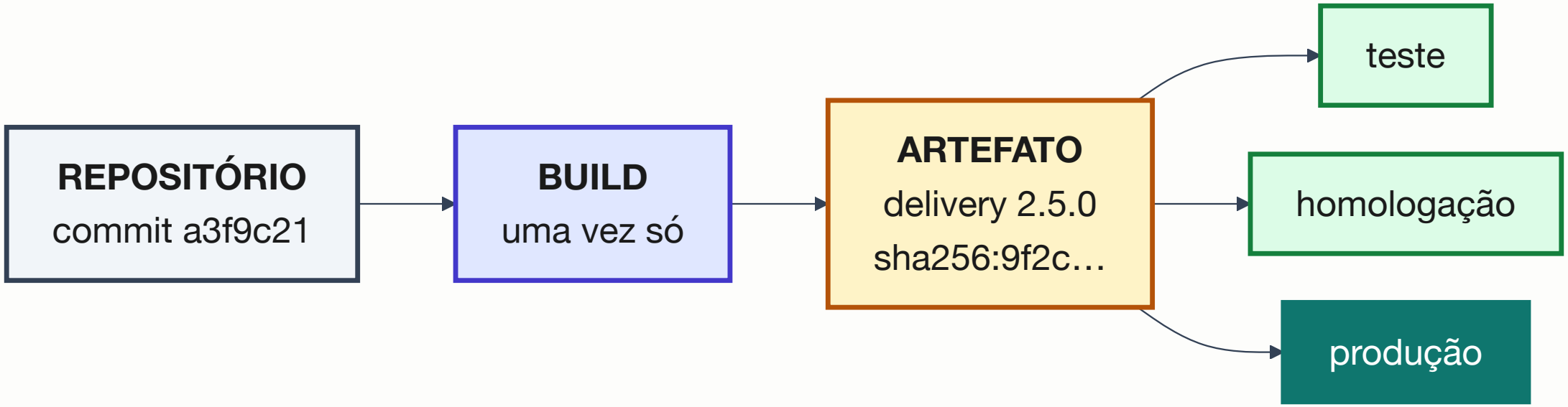
as dependências guardadas pela empresa, para não depender de um serviço de fora

SBOM

a lista de materiais do software: *"nós usamos isso?"* com resposta em segundos

💡 **O build também é por onde se ataca.** Em 2024, uma porta dos fundos no **xz**, biblioteca de quase todo Linux, estava no pacote de release e em arquivos de teste ativados no build — não no código do repositório. Foi achada porque o SSH ficou **meio segundo** mais lento. Construir a partir do repositório, por um processo que qualquer um repete, é parte da defesa.

Construa uma vez



O build produz um **artefato** — identificado pelo hash — e é **ele** que vai a todos os ambientes. Construir de novo para cada um é dar ao sistema outra chance de sair diferente. Os ambientes são a Aula 11.

O pipeline do delivery, a cada pull request

⚡ **gatilho**	cada pull request, e cada integração no `main`	—
🔎 **verificações rápidas**	há senha no código? O estilo segue o padrão do time?	25 s
🔨 **build**	o sistema se constrói do zero, a partir do repositório?	1 min
🧪 **testes de unidade**	cada regra, isolada, faz o que deveria?	40 s
🔗 **testes de integração**	as peças funcionam juntas — banco, pagamento simulado?	4 min
🛡️ **dependências**	alguma biblioteca tem falha de segurança conhecida?	30 s
📦 **artefato**	o pacote verificado, pronto para a Aula 11	✓

Se um estágio falha, os seguintes não rodam — e a mudança fica **vermelha**.

Barato e rápido primeiro — e dez minutos no total



Falhar cedo: o mais barato e o mais provável de falhar vão na frente; o que não depende de nada roda em paralelo. Total: **6,5 min** — a XP propôs **10**.

> ⚠️ **Pipeline de 40 minutos:** ninguém espera o resultado, e o time passa a **juntar mudanças** para economizar execução — o contrário da integração contínua.

O pipeline é um arquivo — versionado como o resto

```yaml
on: [pull_request, push]                # QUANDO: a cada pull request e a cada envio
permissions: { contents: read }         # o mínimo: só ler o código
jobs:
  verificar:
    runs-on: ubuntu-latest              # ONDE: uma máquina nova, criada para isto
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1   # v7.0.1
      - run: npm ci                     # as versões exatas do lockfile
      - run: npm run segredos           # o mais barato primeiro
      - run: npm run lint
      - run: npm run build              # constrói do zero
      - run: npm test                   # unidade e integração
      - run: npm audit --audit-level=high
```

Item de configuração da Aula 09: quem muda este arquivo muda o que conta como *"não quebrou"*.

A pirâmide de testes



↑ No topo

frágeis, caros de escrever, demorados — e, quando falham, não dizem onde

↓ Na base

rápidos, baratos — e apontam a função exata

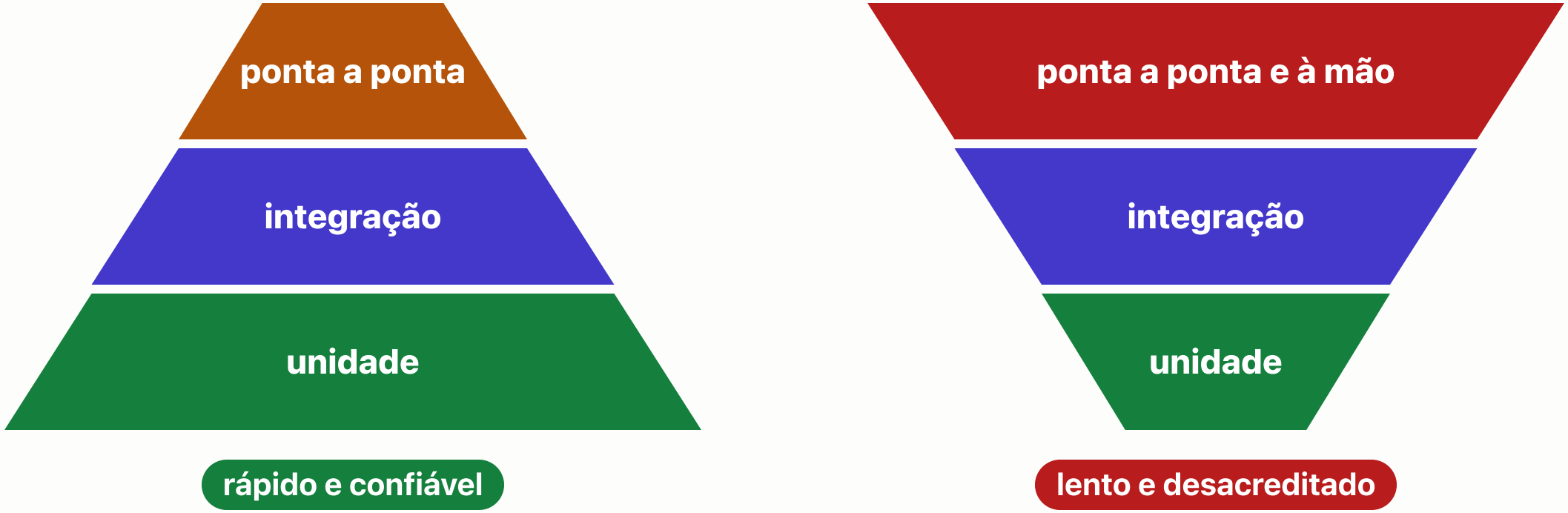
Mike Cohn, *Succeeding with Agile* (2009). A pirâmide diz a **proporção** — não que um nível substitui o outro.

Cada nível vê uma coisa

Nível	Verifica	No delivery
unidade	uma regra, isolada	o frete de 9,5 km custa R$ 12,00
integração	as peças juntas	o pedido é gravado, e o pagamento simulado o confirma
ponta a ponta	o sistema inteiro, como o cliente usa	no navegador: monta o pedido, paga, vê "confirmado"

No sábado: unidade do frete **passou** · unidade do pagamento **passou** · integração dos dois **falhou**
O defeito estava no **encontro** — e só o meio da pirâmide o pega.

A pirâmide e o cone de sorvete



O **cone** é a pirâmide de cabeça para baixo: o pipeline fica lento, falha por motivos que não são defeito — e, com o tempo, ninguém mais confia nele.

O teste instável

Passa e falha **com o mesmo código**. Os números do Google, em 2016:

16%

dos testes mostravam algum grau de instabilidade

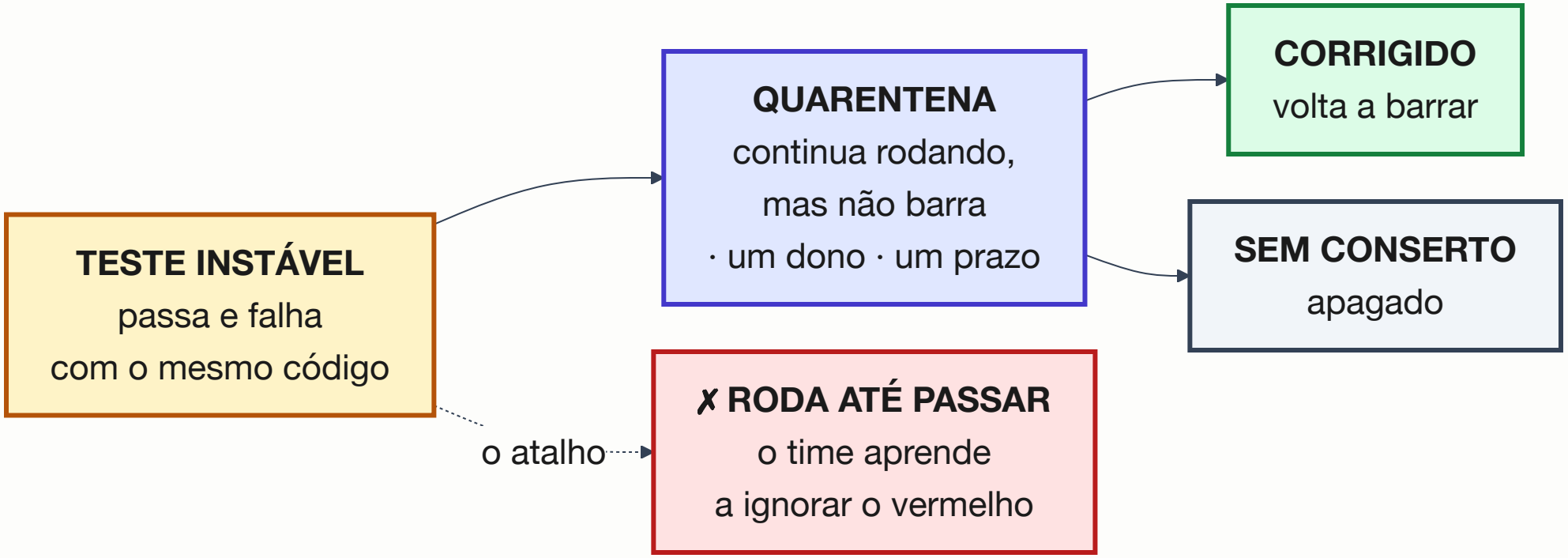
1,5%

de todas as execuções davam resultado instável

As causas de sempre: **o relógio** **a ordem dos testes** **a rede** **"espera 2 s e confere"**

⚠️ **O perigo não é a falha: é o hábito.** *"Falhou? Roda de novo."* — e, no dia em que a falha for de verdade, ela passa por acaso.

Quarentena, não "roda até passar"



Na quarentena, o teste **continua rodando** e deixa de barrar a integração — com **um responsável e um prazo**. No fim do prazo, é corrigido ou apagado.

O pull request que não pode entrar

Pull request #187 — feat: cupom de desconto

✅	verificar / verificações rápidas	22s
✅	verificar / build	1m 02s
✅	verificar / testes de unidade	41s
❌	verificar / testes de integração `pedido com cupom e pagamento on-line: esperado "confirmado", obtido "pendente"`	3m 48s
✅	revisão: aprovado por Carla	

⛔ **Integração bloqueada: 1 verificação obrigatória falhou**

A pessoa aprovou — e a integração continua bloqueada. O **ramo protegido** é o controle de mudanças da Aula 09, executado pela máquina.

O que barra, e o que só avisa

⛔ Barra — é objetivo

- o build falhou
- algum teste falhou
- há senha ou token no código
- biblioteca com falha grave conhecida
- falta a aprovação de outra pessoa

💬 Avisa — pede julgamento

- a cobertura do código alterado caiu
- uma função ficou mais complexa
- há versão nova de uma biblioteca
- o pipeline ficou mais lento

A máquina confere o objetivo; a pessoa revisa a intenção e o desenho. É a parte da **Definição de Pronto** da Aula 06 que se responde com sim ou não — o **sistema de qualidade** ficando executável.

14/03/2025: o pipeline tinha as chaves

a peça

`tj-actions/changed-files`, em mais de 23 mil repositórios

→

a chave

atacantes têm a credencial da conta que o mantém

→

as tags

o código muda, e `v45` e as outras **são movidas**

→

os registros

os pipelines escrevem as senhas nos logs — públicos

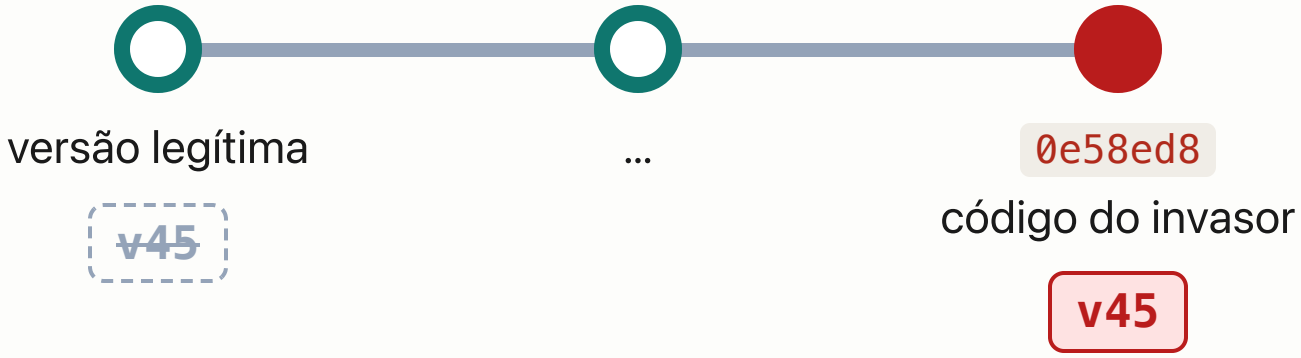
→

a resposta

alerta da CISA: trocar todo segredo exposto

Nenhum dos repositórios afetados mudou uma linha. Bastou o pipeline baixar o componente pela tag, como sempre.

A tag se move; o hash do commit, não



Fixar pelo hash

calculado do conteúdo, não aponta para outra coisa — a versão vai no comentário

Permissão mínima

o pull request só lê o código; segredo só no estágio que precisa

Segredo exposto se troca

apagar o log não desfaz o vazamento — a lição da Aula 09

Cobertura: executado não é verificado

```
// teste A — executa e confere
test("frete de 9,5 km custa R$ 12", () => {
  expect(calcularFrete(9.5)).toBe(12.00);
});
```

cobertura **100%**

quebrou o arredondamento? **falha**

```
// teste B — executa e não confere
test("calcula o frete", () => {
  calcularFrete(9.5);
});
```

cobertura **100%**

quebrou o arredondamento? **continua passando**

> 💡 **Teste de mutação: testar os testes.** A ferramenta troca um `>` por `>=`, um `+` por `-`, e roda a suíte. O teste B deixa **todos os mutantes vivos**.

Quando a medida vira meta

> **Quando uma medida vira meta, ela deixa de ser uma boa medida.**
> — Marilyn Strathern (1997), a partir de Charles Goodhart (1975)

a regra
"abaixo de 90% de cobertura, não entra"

→

o atalho
testes como o B, trechos triviais, arquivos difíceis fora da conta

→

o resultado
o número sobe em duas semanas; os defeitos, iguais

Sem má-fé: é o comportamento racional diante de uma barreira numérica. É a **velocidade usada para cobrar**, da Aula 06.

A cobertura como lanterna

60%
aceitável

75%
louvável

90%
exemplar

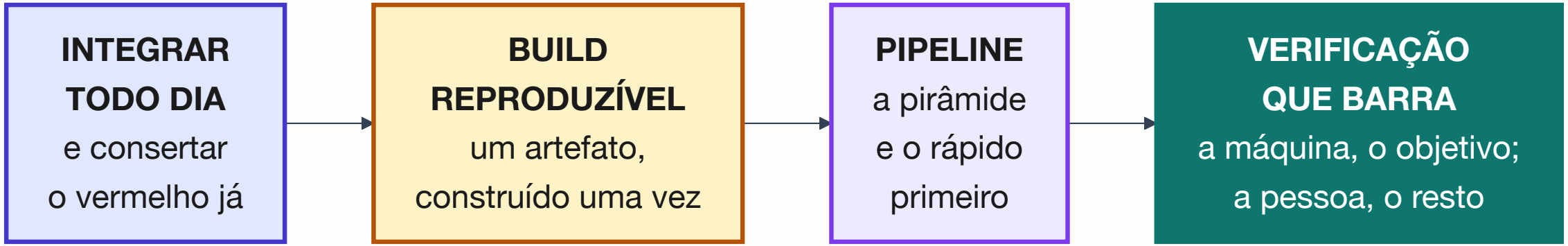
Referências do Google (2020) — que diz que **não existe número ideal** e prefere que cada time escolha a sua.

No pull request
a cobertura do **código alterado**, para quem revisa

No risco
o que está com zero no pagamento, ou no acesso ao prontuário

Avisar quando cai
em vez de barrar abaixo de um número

O que esta aula deixa no lugar



O artefato que sai verde está **pronto** para ir ao ar. Estar pronto e estar no ar são coisas diferentes.

➡️ Próxima aula

Aula 11 — Entrega e implantação contínuas

O artefato está pronto.

Quem decide quando ele vai ao ar —

e para quantos clientes de cada vez?